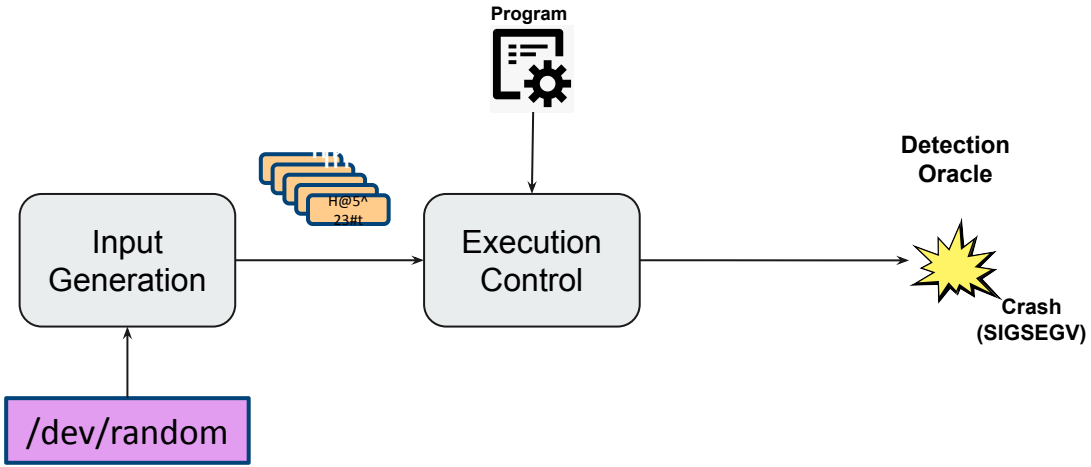
Dynamic Analysis - Fuzzing

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

Fuzzing: Gen 1 (Random data)



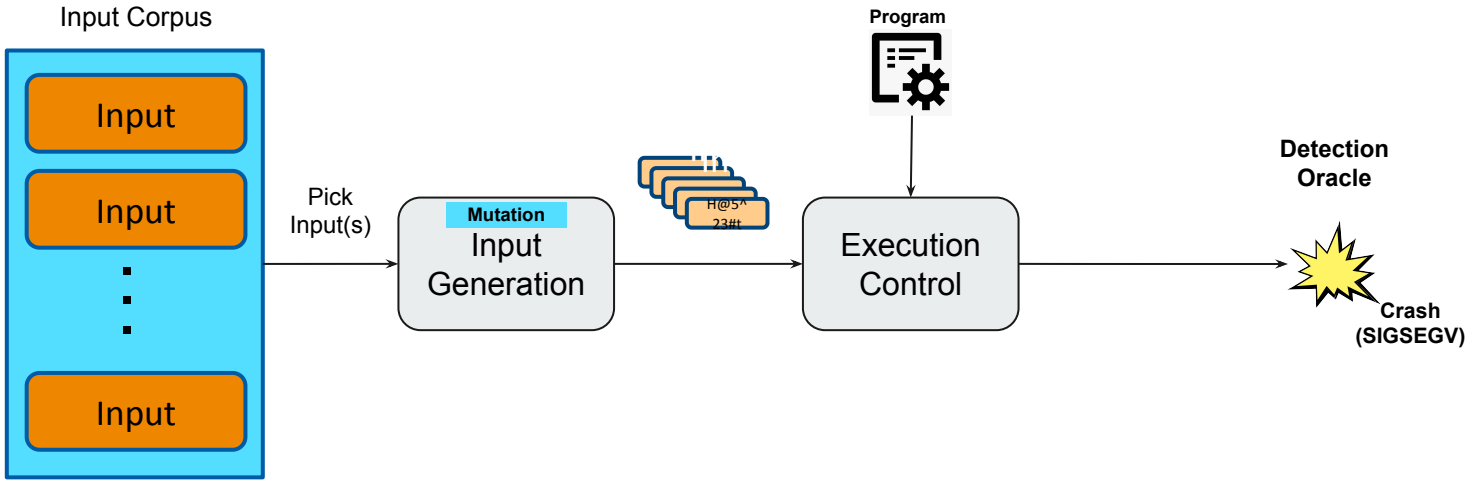
Fuzzing: Gen 1

- Conducted by Barton Miller @ Univ of Wisconsin.
- 1990: Command-line fuzzer, testing reliability of UNIX programs.
  - Bombards utilities with random data
- 1995: Expanded to GUI-based programs (X Windows), network protocols, and system library APIs.
- Later: Command-line and GUI-based Windows and OS X apps.

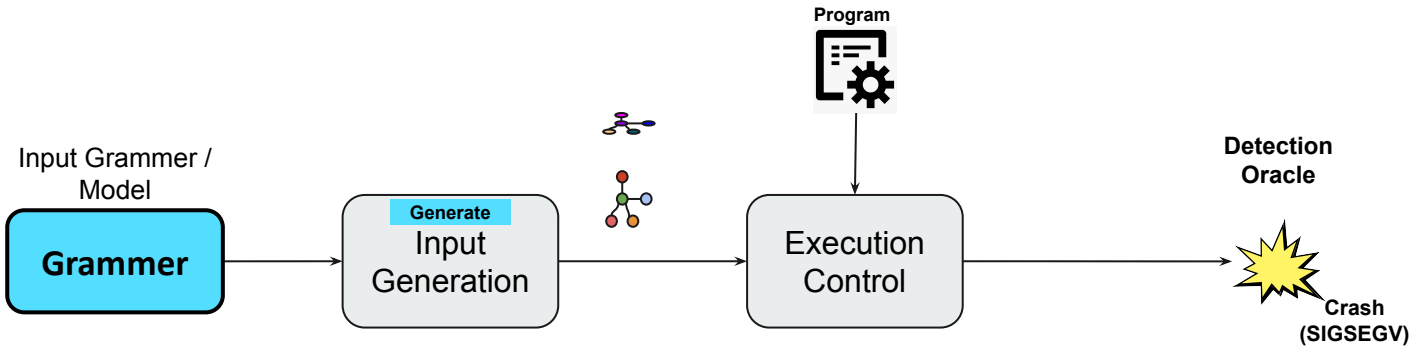
Caused 25-33% of UNIX utility programs to crash (dump state) or hang (loop indefinitely).

- Hard to generate well formed data:
  - E.g., PNG files.

Fuzzing: Gen 2.a (Mutation based)



Fuzzing: Gen 2.b (Generation based)

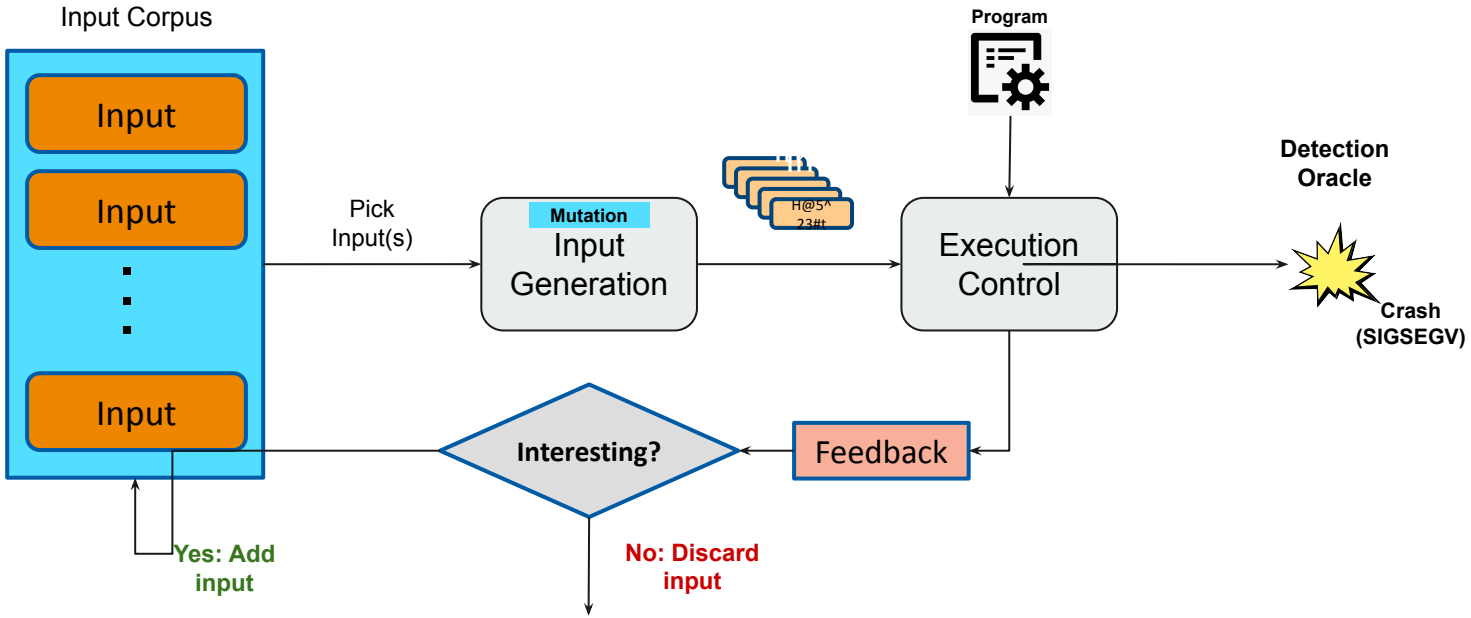


Fuzzing: Gen 2.b

- Very effective at generating complex inputs:
  - Csmith: Generate syntactically valid but random C programs.
- Commercial tools:
  - Peach. 

- Need to manually write these input grammars:
  - Domain Specific Language.
  - Large: ~200 lines

Fuzzing: Gen 3 (Feedback guided Mutation based)



Fuzzing: Gen 3

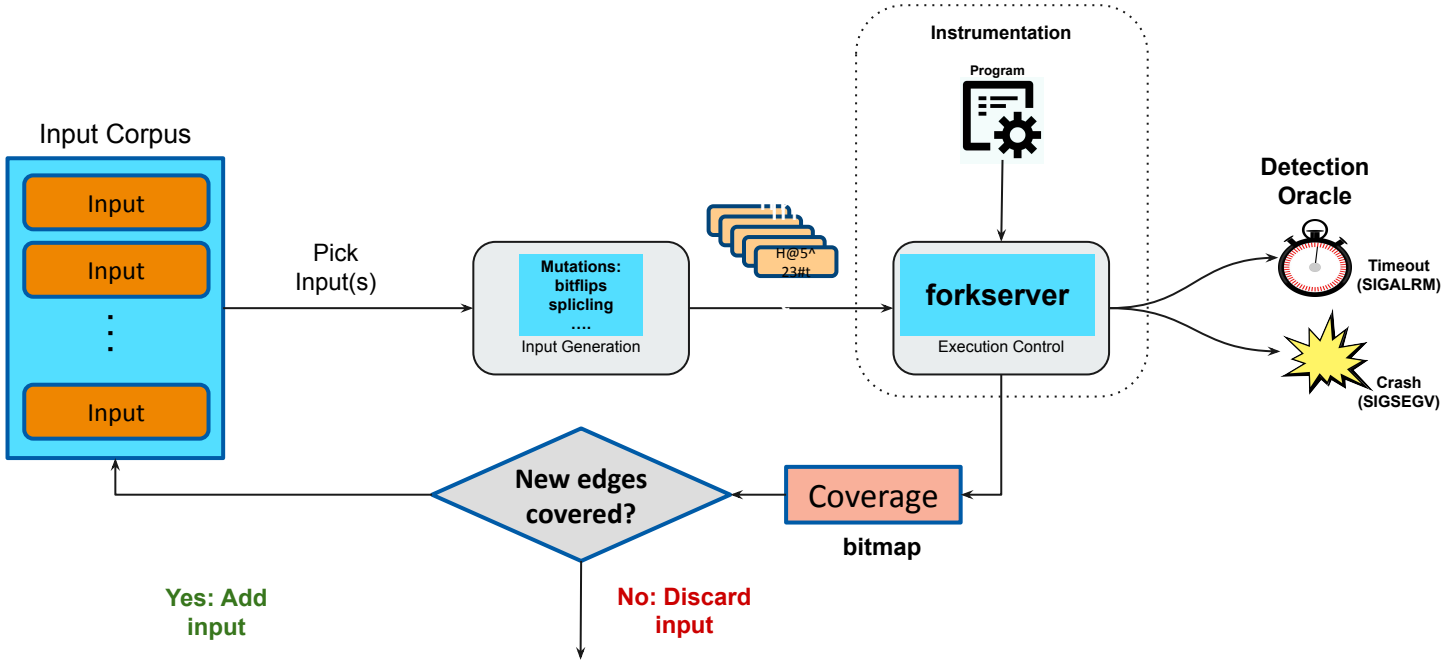
- Extremely effective at quickly generating well-formed inputs.
- Highly successful commercial grade tool:
  - AFL (AFLPlusPlus)
- Need a way to capture feedback: Impacts performance.



Fuzzer Deep Dive : AFLPlusPlus

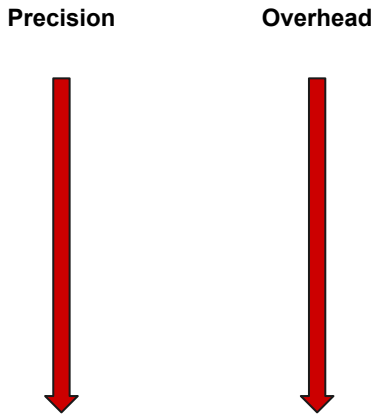
- Based on American Fuzzy Lop (AFL) developed by Michał Zalewski
- Coverage Feedback based Mutational Fuzzing.
- Highly customizable, efficient and very well maintained.
- Revolutionized fuzzing research:
  - ~50 papers since 2015.
- Found various (~1000) bugs in well-maintained programs.

AFLPlusPlus (A++): Coverage guided



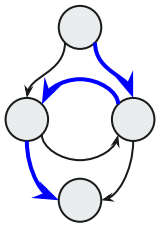
A++: Coverage Map

- Coverage choices:
  - Line or Basic block coverage:
    - Basic blocks executed.
  - **Edge coverage (used by A++):**
    - **Edges (Basic block tuple) executed.**
  - Path coverage:
    - Sequence of basic blocks executed.

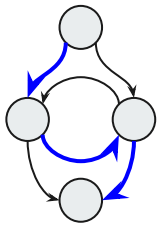


Coverage choices

Execution 1



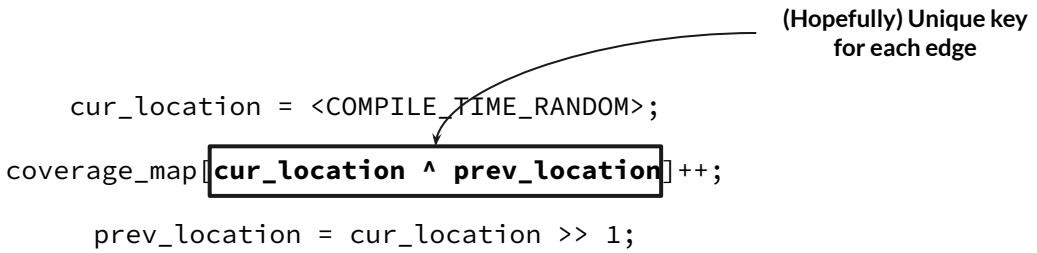
Execution 2



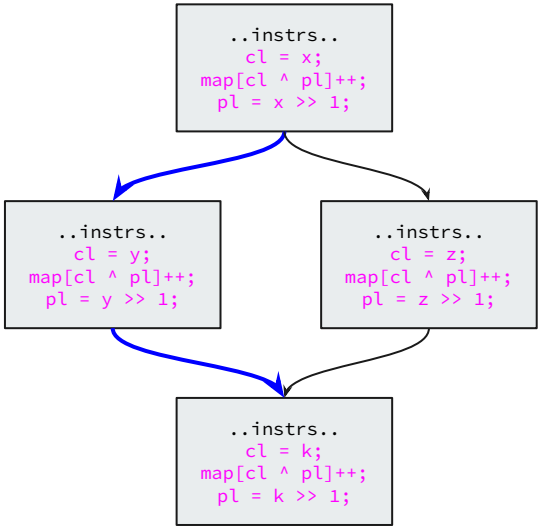
- Does Execution 1 and 2 have:
  - Same basic block coverage?
  - Same edge coverage?
  - Same path coverage?

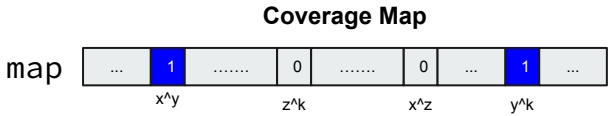
A++: Coverage Instrumentation

- Coverage map: Memory area in the program that stores coverage.
- Every basic block in all functions will be instrumented to update coverage map.

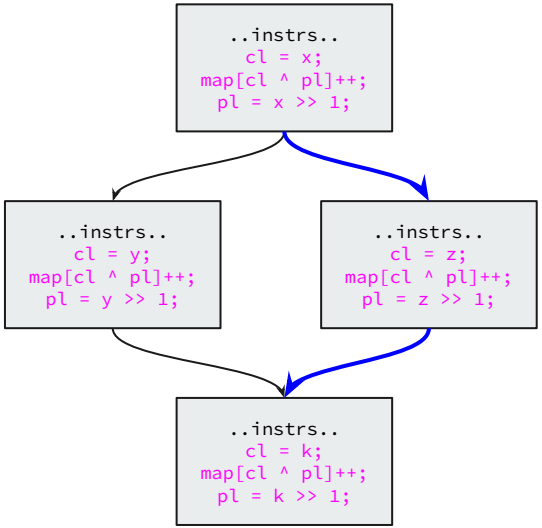


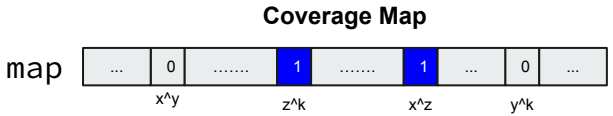
A++: Coverage Map



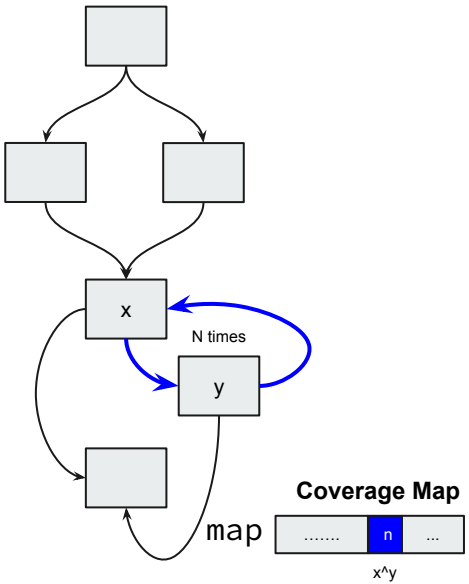


A++: Coverage Map

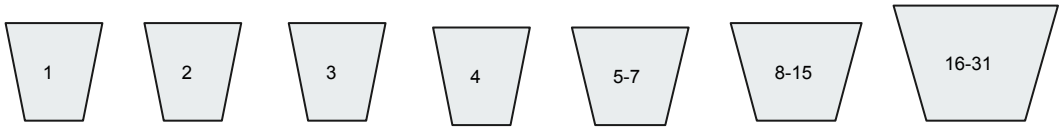




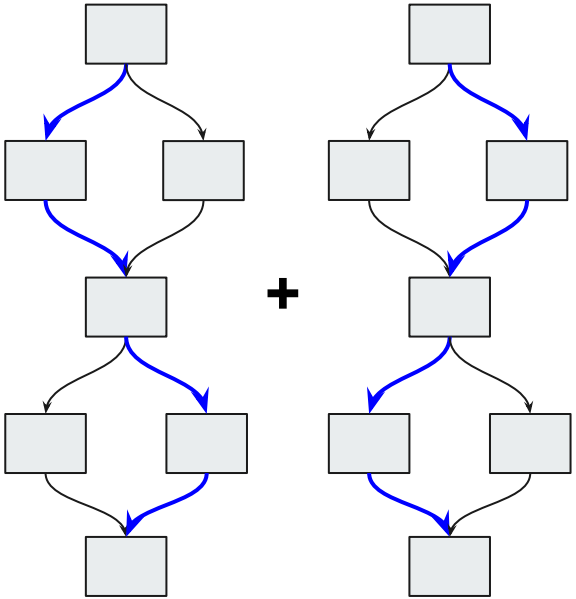
A++: Coverage Map: Bucketized edge counts



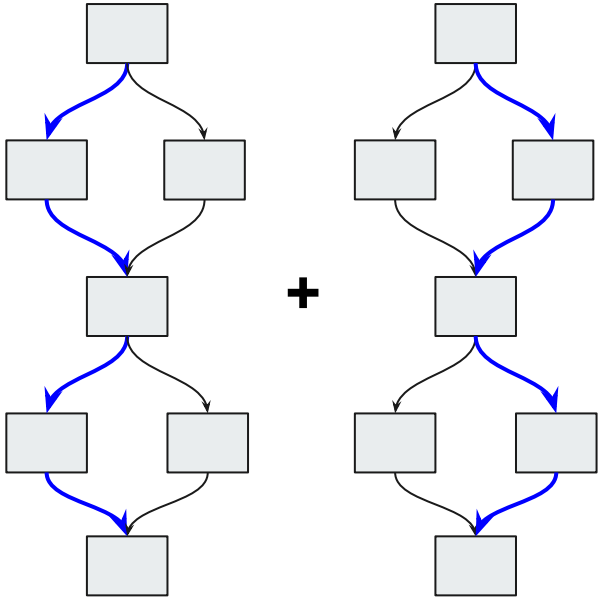
- Edge counts are bucketized:
  - E.g., Coverage map of executions with loop counts that belong to the same bucket will be considered the same.



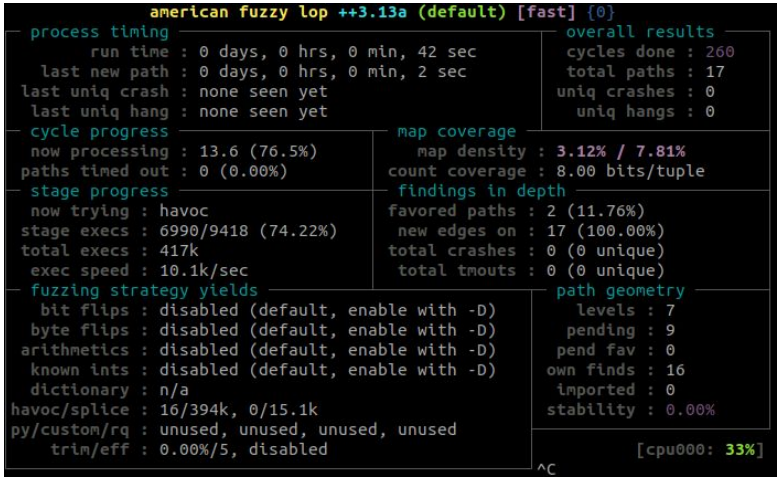
A++: Coverage Map



Has same coverage map as



Using A++

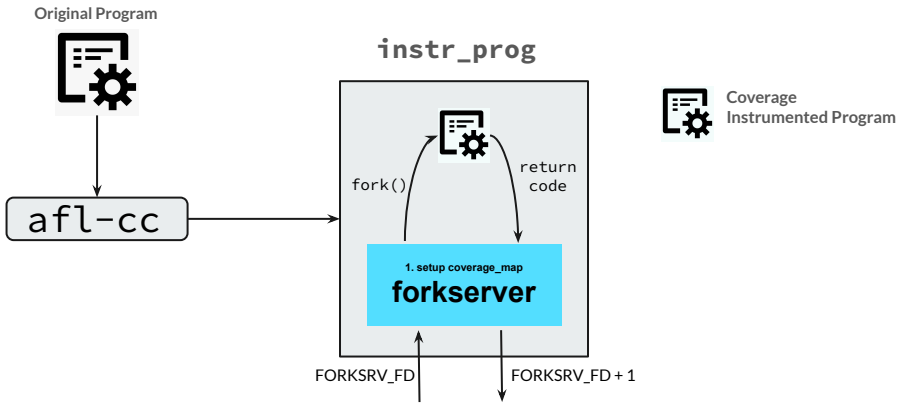


- Instrumentation: Compile the target program using afl compiler, i.e., `afl-cc`:
  - `afl-cc <.c> -o instr_prog`
  - Does:
    - Instrumentation to compute coverage.
    - Add forkserver.

- Fuzzing: Start fuzzing `instr_prog`:
  - `afl-fuzz -i <inputs_folder> -o <output_folder> -- instr_prog`

A++: Instrumentation

`afl-cc <.c> -o instr_prog`



- forkserver (`constructor`) `afl-compiler-rt.o.c`:

  Setup coverage map (shared memory map).

  while (1) {

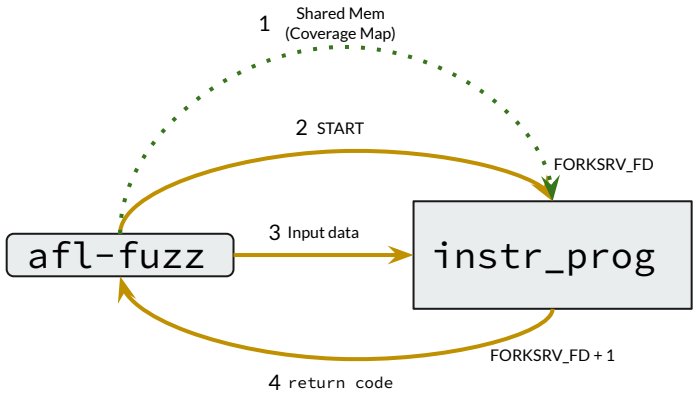
  1. Wait for command at FORKSRV_FD.
  2. Once received, `fork` and start executing main of the original program:

     Program would be writing to coverage map (shared memory).
  1. Sends the return code through FORKSRV_FD + 1

  }

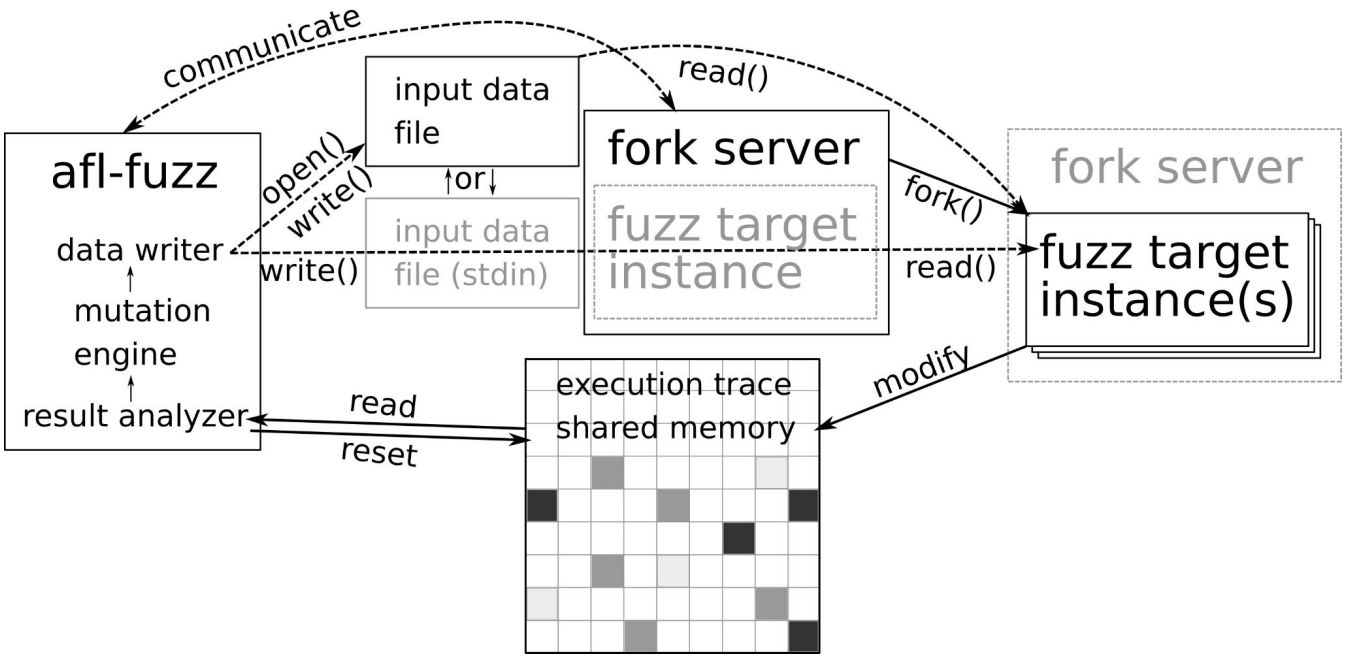
A++: Fuzzing

```
afl-fuzz -i <inputs_folder> -o <output_folder> -- instr_prog
```



- afl-fuzz (`src/afl-fuzz.c`):
  - Setup (1):
    - Send shared memory id.
  - Fuzzing Loop (2-3-4):
    - 2. START
    - 3. Input data (stdin or file)
    - 4. return code (crashes or timeout)

A++: Fuzzing


communicate
read()
input data file
fork server
afl-fuzz
open()
write()
or
input data file (stdin)
fuzz target instance
fork server
fork()
data writer
write()
read()
fuzz target instance(s)
mutation engine
result analyzer
read
reset
execution trace shared memory
modify